

Claude Code — Quick Reference Cheat Sheet

Windows Users: WSL Required

Claude Code runs on Linux/macOS. On Windows, you need **WSL (Windows Subsystem for Linux)**.

```
# In PowerShell (as Administrator):  
wsl --install  
  
# Restart your PC, then open "Ubuntu" from Start menu  
# You're now in a Linux terminal – all commands below run here
```

Important: Always work inside WSL, not in PowerShell/CMD. Your projects live in the Linux filesystem (e.g., `~/projects/`), not on `/mnt/c/`. This avoids performance issues and path problems.

Prerequisites (WSL/Ubuntu)

macOS users: Replace `apt` commands with `brew install` equivalents. See the [full guide](#) for macOS-specific commands.

```
# Update system packages first  
sudo apt update && sudo apt upgrade -y  
sudo apt install -y build-essential curl wget git  
  
# Install Node.js (v18+) – needed for Codex and Gemini CLIs  
curl -fsSL https://deb.nodesource.com/setup_lts.x | sudo -E bash -  
sudo apt-get install -y nodejs  
  
# Install GitHub CLI  
curl -fsSL https://cli.github.com/packages/githubcli-archive-keyring.gpg \  
| sudo dd of=/usr/share/keyrings/githubcli-archive-keyring.gpg  
echo "deb [arch=$(dpkg --print-architecture) \  
signed-by=/usr/share/keyrings/githubcli-archive-keyring.gpg] \  
https://cli.github.com/packages stable main" \  
| sudo tee /etc/apt/sources.list.d/github-cli.list > /dev/null  
sudo apt update && sudo apt install -y gh  
  
# Install Claude Code (native installer – recommended, auto-updates)  
curl -fsSL https://claude.ai/install.sh | bash  
  
# Install Codex CLI and Gemini CLI  
npm install -g @openai/codex  
npm install -g @google/gemini-cli  
  
# Configure git  
git config --global user.name "Your Name"  
git config --global user.email "your@email.com"  
git config --global core.autocrlf input # WSL line-ending fix
```

```
# Authenticate GitHub
gh auth login
```

Tool	What it is	Account / API Key Needed
Claude Code	AI coding agent — Claude Opus/Sonnet	Anthropic API key or Claude Max subscription
Codex CLI	OpenAI's coding agent — for code review	OpenAI API key or ChatGPT Plus/Pro/Team subscription
Gemini CLI	Google's AI CLI — for code review	Google account (free tier available)
GitHub CLI	Manage repos, PRs, issues from terminal	GitHub account (via <code>gh auth login</code>)

Model knowledge cutoff: Claude, Codex, and Gemini all have training data cutoff dates. They may not know the latest library versions or APIs. Always specify “use the latest stable version” for dependencies.

Every New Project Starts Here

Three steps. This is all you need to remember.

Step 1: Create a directory and start Claude

```
mkdir ~/projects/my-project && cd ~/projects/my-project
claude --dangerously-skip-permissions
```

Step 2: Paste this as your first prompt (edit the bracketed parts)

Read the full guide and cheat sheet in this repo and internalize the workflow, best practices, and conventions described:
<https://github.com/daveremy/claude-code-guide>

This is my development workflow — follow it for this entire project.

Key points:

- Always plan before coding (plan mode)
- I will review plans with Codex and Gemini before you implement
- After implementation, I will run `/simplify` and do multi-AI code reviews
- Use git with frequent commits
- Maintain `CLAUDE.md`, `README.md`, and `ROADMAP.md` (for bigger projects)
- Use latest stable versions of all dependencies
- Documentation is critical — keep it updated

Now, here is what I want to build: [YOUR PROJECT DESCRIPTION]

Features:

- [feature 1]
- [feature 2]
- [feature 3]

Tech stack: [your preferences, or "suggest appropriate tech"]

Please enter plan mode and create an implementation plan.

Step 3: Follow the workflow (see below)

Tip: If you clone this repo locally (`git clone https://github.com/daveremy/claude-code-guide.git ~/claude-code-guide`), you can reference the files directly instead of the URL: Read `~/claude-code-guide/claude-code-guide.md` and `~/claude-code-guide/claude-code-cheatsheet.md` – internalize the workflow and follow it for this project.

Essential Claude Code Commands

Command	What it does
<code>claude</code>	Start Claude Code in current directory
<code>claude --dangerously-skip-permissions</code>	Start without permission prompts
<code>/plan</code>	Enter plan mode (design before coding)
<code>/commit</code>	Commit staged changes with AI-generated message
<code>/simplify</code>	Run code simplifier — finds improvements
<code>/help</code>	Show all available commands
<code>Ctrl+C</code>	Cancel current operation
<code>Escape</code>	Back out / cancel

The Workflow (memorize this)

1. DESCRIBE → Write project/feature description, ask for plan mode
2. PLAN → Claude creates implementation plan
3. REVIEW → Get AI reviews from Codex + Gemini on the plan
4. LOOP → Revise plan until all reviewers approve
5. BUILD → Claude implements the approved plan
6. SIMPLIFY → Run `/simplify`, loop until clean
7. REVIEW → AI code reviews with Codex + Gemini
8. LOOP → Revise code until all reviewers approve
9. TEST → Ensure tests exist and pass
10. VERIFY → Manual testing with provided instructions
11. COMMIT → Commit with clear message

Git Practices

```
# Commit early and often
# Use clear, descriptive commit messages
# Create a repo on GitHub early
git init
```

```
gh repo create my-project --public --source=. --push
```

```
# Good commit rhythm:
#   - After plan is approved
#   - After each feature/component works
#   - After code review fixes
#   - After tests pass
```

Key Files

File	Purpose
CLAUDE.md	Instructions for Claude — project context, conventions, rules
README.md	Project documentation — also helps AI understand your project
ROADMAP.md	For bigger projects — tracks phases and progress

CLAUDE.md Starter Template

```
# Project Name

## Description
[What this project does in 2-3 sentences]

## Tech Stack
- [Language/framework]
- [Key dependencies – specify versions if needed]

## Conventions
- [Coding style preferences]
- [File organization rules]

## Rules
- Use latest stable versions of all dependencies
- Write tests for all new features
- Keep functions small and focused
```

Prompt Tips

Do this	Not this
“Build a REST API with Express that handles user CRUD operations”	“Build an API”
“Enter plan mode and design a todo app with React and local storage”	“Make me a todo app”
“Use the latest stable version of Next.js”	“Use Next.js” (may use outdated version)
“Break this into 3 phases: auth, dashboard, settings”	Trying to build everything at once

When AI Doesn't Know Something

- **Outdated dependency info** → Tell Claude: “use latest stable version of X”
 - **New API/framework features** → Use Context7: Claude can fetch live docs
 - **Conflicting advice from reviewers** → You make the final call
 - **Hallucinated APIs** → Always verify with official docs if something looks off
-

Big Project Strategy

Phase 1: Core functionality → build, review, commit
Phase 2: Secondary features → build, review, commit
Phase 3: Polish & edge cases → build, review, commit
Phase 4: Testing & documentation → test, document, commit

Update ROADMAP.md after completing each phase.

WSL Tips

- **Always work in the Linux filesystem** (~/projects/), not /mnt/c/ — much faster
 - **VS Code integration**: Install “Remote - WSL” extension, then `code .` from WSL opens VS Code connected to WSL
 - **Copy/paste in terminal**: `Ctrl+Shift+C` / `Ctrl+Shift+V` (not `Ctrl+C` / `Ctrl+V`)
 - **Access Windows files if needed**: They're at `/mnt/c/Users/YourName/`
 - **If npm permissions fail**: `sudo chown -R $(whoami) ~/.npm`
-